

Contrato de Servicio — Catálogo (Equipo B)

Versión: 2.2 — Segunda entrega (Ciclo 2) **Fecha:** 2026-09-10 **Equipo responsable:** Equipo B **Consumido por:** Equipo A (Búsqueda), Equipo C (Carrito)

1. Propósito

Catálogo es la fuente de verdad de los productos de la tienda. Expone operaciones CRUD vía REST (a través de Kong) y publica eventos a RabbitMQ para que otros servicios se mantengan sincronizados sin tener que consultarlo en cada operación.

2. Endpoints REST

Se consumen a través del API Gateway (Kong). Base path propuesto: `/api/catalogo` (ajustable cuando se configure Kong).

Método	Endpoint	Descripción	Auth
GET	/productos	Lista productos activos, paginado, filtro opcional <code>?categoria=</code>	Pública
GET	/productos/{id}	Detalle de un producto	Pública
GET	/categorias	Lista las categorías válidas	Pública
POST	/productos	Crea un producto	Admin
PUT	/productos/{id}	Actualiza un producto	Admin
DELETE	/productos/{id}	Desactiva un producto (soft delete)	Admin

Identity Server (Keycloak) no está integrado en esta entrega. Los endpoints Admin no validan token real todavía — se documenta el contrato final para que Carro y Búsqueda ya sepan qué esperar cuando se conecte.

GET /productos → 200

```
{
  "productos": [ { "...": "ver modelo Producto en sección 3" } ],
  "total": 42,
  "pagina": 1,
  "tamanoPagina": 20
}
```

Parámetros de consulta (todos opcionales, se pueden combinar):

Parámetro	Por defecto	Reglas
categoria	todas	id de una categoría (ver GET /categorias)
pagina	1	entero ≥ 1 — la primera página es la 1
tamanoPagina	20	entero entre 1 y 100

Ejemplo: GET /productos?categoria=cat-ropa&pagina=2&tamanoPagina=20

- total es la cantidad de productos activos que cumplen el filtro, no solo los de la página actual.
- Si pagina es mayor que la última página, responde 200 con productos: [] y el mismo total.
- Si categoria no corresponde a ninguna categoría existente, responde 200 con productos: [] y total: 0 (un filtro sin resultados no es un error).
- Valores de pagina o tamanoPagina fuera de estas reglas → 400 (VALIDACION_FALLIDA).

GET /productos/{id} → 200 (Producto) · 404 (PRODUCTO_NO_ENCONTRADO)

Devuelve el producto aunque esté inactivo (viene con activo: false). El 404 es solo para un id que no existe.

GET /categorias → 200

```
[
  { "id": "cat-ropa", "nombre": "ropa" },
  { "id": "cat-hogar", "nombre": "hogar" },
  { "id": "cat-electronica", "nombre": "electrónica" }
]
```

POST /productos → 201 (Producto creado, con id asignado) · 400 (VALIDACION_FALLIDA)

PUT /productos/{id} → 200 (Producto actualizado) · 404 · 400

Reemplaza el producto completo: el cuerpo lleva todos los campos, con las mismas reglas que POST (sección 3).

- No cambia id (se toma de la URL) ni activo (solo lo cambia DELETE); si vienen en el cuerpo, se ignoran.
- Sobre un producto inactivo responde 200 y el producto sigue inactivo.

DELETE /productos/{id} → 204 (marca activo: false, no borra el registro) · 404

Si el producto ya estaba inactivo, responde 204 sin cambiar nada ni publicar otro evento (la operación es idempotente).

3. Modelo de Producto

Campo	Tipo	Descripción
id	string	Identificador único (generado por Mongo)
nombre	string	Requerido, máx. 120 caracteres
descripcion	string	Opcional
precio	number	Requerido, > 0
categoria	string	Requerido, debe coincidir con un <code>id</code> de <code>GET /categorias</code>
stock	integer	Requerido, >= 0
imagenes	array[string]	Opcional, URLs
activo	boolean	<code>true</code> al crearse; <code>false</code> tras un soft delete. Solo lo cambia <code>DELETE</code>

Ejemplo:

```
{
  "id": "64f1a2b3c9e77a001f3d8e21",
  "nombre": "Camiseta básica algodón",
  "descripcion": "Camiseta unisex 100% algodón, corte regular.",
  "precio": 49900,
  "categoria": "cat-ropa",
  "stock": 120,
  "imagenes": ["https://cdn.tienda.com/img/camiseta-1.jpg"],
  "activo": true
}
```

Modelo de Categoría

Campo	Tipo	Descripción
id	string	Identificador único, usado como valor de <code>Producto.categoria</code>
nombre	string	Nombre visible de la categoría

En esta entrega las categorías vienen precargadas (seed) — ver sección 8.

4. Formato de error estándar

```
{
  "codigo": "PRODUCTO_NO_ENCONTRADO",
  "mensaje": "descripción legible del error"
}
```

Códigos de esta entrega: `PRODUCTO_NO_ENCONTRADO` (404), `VALIDACION_FALLIDA` (400).

5. Eventos publicados (RabbitMQ)

Catálogo publica en un exchange de RabbitMQ de tipo **topic** llamado `catalogo.eventos`. Cada evento sale con su routing key; cada consumidor crea su propia cola y la enlaza al exchange con las routing keys que le interesan (por ejemplo, `producto.*` para recibirlos todos). El cuerpo del mensaje es un JSON con los campos del payload.

Evento	Routing key	Se publica cuando	Payload mínimo
ProductoCreado	<code>producto.creado</code>	se crea un producto	id, nombre, precio, categoria
ProductoActualizado	<code>producto.actualizado</code>	se modifica cualquier campo	id, nombre, precio, categoria
ProductoDesactivado	<code>producto.desactivado</code>	se hace soft delete (no se repite si el producto ya estaba inactivo)	id

6. Integración con Equipo A (Búsqueda)

Búsqueda **no debe** llamar a `GET /productos` para construir su índice. Debe suscribirse a los eventos de la sección 5 y mantener su propio índice de forma asíncrona.

Pendiente de confirmar con A: si necesitan `descripcion` en el payload del evento para full-text search (hoy no está incluida).

7. Integración con Equipo C (Carrito)

Antes de agregar un producto al carrito, Carro debe llamar a `GET /productos/{id}` y validar:

- Que el producto existe (si no, 404 `PRODUCTO_NO_ENCONTRADO`)
- Que `activo == true`
- Usar el `precio` que devuelve Catálogo — nunca el que mande el frontend

Pendiente de confirmar con C: qué hace Carro si un producto se desactiva después de estar en el carrito.

8. Fuera de alcance en esta entrega

- Autenticación real vía Keycloak
- Descuento y Órdenes (equipo aún sin asignar)
- Variantes de producto (talla/color con stock por variante)
- Filtros y ordenamiento avanzado (solo se filtra por categoría en esta entrega)
- Gestión de categorías (`POST` / `PUT` / `DELETE` / `/categorias`) — las categorías vienen precargadas por ahora, solo se listan

9. Frontend — módulo Catálogo

El módulo "Catálogo" del Cliente Web (dentro del Host App) consume estos endpoints directamente, a través de Kong:

Vista	Endpoint que consume	Qué muestra
Listado de productos	<code>GET /productos</code>	Grid o lista: nombre, precio, imagen, categoría
Detalle de producto	<code>GET /productos/{id}</code>	Vista individual con toda la información
Crear producto (admin)	<code>POST /productos</code>	Formulario: nombre, descripción, precio, categoría, stock, imágenes
Selector de categoría	<code>GET /categorias</code>	Dropdown para el formulario de creación

Framework: Vue.js, acordado por los 3 equipos para todos los módulos del Host App, de modo que sean consistentes entre sí.

Pendiente de decidir: cómo se integra cada módulo al Host App (Module Federation u otro mecanismo).

No necesita ser un diseño elaborado: con que consuma los endpoints reales (nada de datos mockeados) y refleje los estados ya definidos (producto inactivo, error 404, etc.) alcanza para el nivel de esta entrega.

10. Historial de cambios

Fecha	Cambio
2026-08-20	v1.0 — versión inicial para reunión con equipos A y C
2026-08-20	v1.1 — se agrega <code>GET /categorias</code> y modelo de Categoría
2026-08-21	v2.0 — se reasigna a Segunda entrega (Ciclo 2); se agrega sección de Frontend

Fecha	Cambio
2026-09-10	v2.1 — Aclaraciones compatibles con v2.0: parámetros de paginación de <code>GET /productos (pagina , tamanoPagina)</code> ; <code>?categoria=</code> inexistente devuelve lista vacía; <code>GET /productos/{id}</code> devuelve inactivos con <code>activo:false</code> ; reglas de <code>PUT</code> (reemplazo completo, no cambia <code>id</code> ni <code>activo</code>) y de <code>DELETE</code> (idempotente); exchange <code>catalogo.eventos</code> y routing keys de los eventos; <code>stock</code> en el formulario de creación; variantes fuera de alcance
2026-09-10	v2.2 — §9: el frontend se construye con Vue.js, acordado por los 3 equipos; sigue pendiente el mecanismo de integración con el Host App